

Dominar Claude de principio a fin

Curso completo: de Claude Chat a Cowork, Claude Code, Skills y MCP

Manual de estudio, práctica y referencia para aprender a trabajar con Claude como asistente, investigador, analista, creador de contenido, entorno de prototipado y agente de trabajo. Pensado para aprendizaje autodidacta, de cero a nivel avanzado.

Cómo nació este curso

Este material amplía y reorganiza el contenido de los *Cursos de Claude de internet* hacia un programa mucho más profundo y autocontenido. Donde el curso original muestra **qué botones tocar**, este manual explica **por qué, cuándo, con qué criterio y qué hacer cuando algo sale mal**. Cada clase original se convierte aquí en una unidad con teoría, ejemplos guiados, ejercicios *con su solución desarrollada* y plantillas listas para usar.

El curso no asume conocimientos previos, pero tampoco se queda en lo básico: termina enseñándote a construir agentes, conectar herramientas y diseñar capacidades reutilizables.

Nota sobre versiones: la tecnología de Claude evoluciona rápido. Los nombres de modelos y funciones de este manual reflejan el estado a mediados de **2026** (Claude Opus 4.7, Claude Sonnet 4.6 y Claude Haiku 4.5). Los **principios** que enseña —cómo pensar, estructurar y evaluar el trabajo con IA— se mantienen aunque cambien las versiones. Verifica detalles puntuales de precios o disponibilidad en la documentación oficial.

A quién va dirigido

- Personas que quieren aprender Claude por su cuenta, de forma ordenada y profunda.
 - Profesionales de cualquier área (datos, marketing, operaciones, RR. HH., producto, educación, derecho, finanzas) que buscan integrar la IA en su trabajo diario.
 - Quienes ya usan asistentes de IA pero todavía improvisan y quieren un método sólido.
 - Curiosos que desean entender de verdad cómo funcionan los agentes, las skills y MCP, no solo usarlos.
- No necesitas saber programar para los primeros cinco pilares. El Pilar 6 (Claude Code) introduce conceptos técnicos, pero está explicado para que cualquiera entienda la idea aunque no escriba código.

Cómo está organizado: los 6 pilares

El curso se estructura en seis pilares que van de lo conceptual a lo operativo.

Pilar	Tema	Qué aprenderás
1. Fundamentos	Entender Claude	Qué es, cómo razona, el ecosistema, qué modelo usar, configuración y seguridad
2. Claude Chat	Conversar con criterio	Prompting profesional, control de salida, iteración, búsqueda web, pensamiento extendido, deep research

Pilar	Tema	Qué aprenderás
3. Trabajo con información	Convertir datos en entregables	Documentos, imágenes, Excel/análisis, presentaciones y artefactos
4. Espacios persistentes	Continuidad real	Proyectos y bases de conocimiento, estilos personalizados, memoria
5. Integración y extensión	Conectar y reutilizar	MCP y conectores, skills personalizadas
6. Claude como agente	Ejecutar trabajo real	Claude Code, Claude Cowork, proyecto integral, mentalidad final

Cada **clase** sigue siempre la misma plantilla, para que puedas estudiar, enseñar o consultar con facilidad:

1. **Introducción** — por qué importa el tema.
2. **Objetivos de aprendizaje** — qué sabrás hacer al terminar.
3. **Conceptos clave** — el vocabulario y las ideas centrales.
4. **Desarrollo** — la explicación a fondo.
5. **Ejemplo guiado** — un caso resuelto paso a paso.
6. **Ejercicio** — para que practiques.
7. **Desarrollo del ejercicio** — la solución comentada (no solo la respuesta).
8. **Errores frecuentes** — qué evitar.
9. **Para llevar** — el resumen accionable.

Objetivos generales del curso

Al terminar serás capaz de:

- Explicar qué es Claude, cómo genera respuestas y cuáles son sus límites reales.
- Elegir el entorno correcto (Chat, Proyectos, Code, Cowork) y el modelo correcto (Haiku, Sonnet, Opus) para cada tarea.
- Diseñar prompts robustos, reutilizables y evaluables.
- Investigar con criterio usando búsqueda web, pensamiento extendido y deep research.
- Analizar documentos, imágenes, hojas de cálculo y datos, y producir informes, presentaciones y aplicaciones.
- Crear espacios persistentes con proyectos, estilos y memoria.
- Conectar Claude a tus herramientas con MCP y construir skills reutilizables.
- Operar Claude como agente con Claude Code y Cowork, con criterios de seguridad, calidad y gobernanza.
- Diseñar un flujo de trabajo completo de principio a fin y medir su valor.

PILAR 1 — Fundamentos: entender Claude

Antes de aprender funciones, hay que entender qué es Claude, cómo "piensa" y qué decisiones tomarás una y otra vez (qué entorno, qué modelo). Este pilar construye la base mental sobre la que se

Clase 1.1 — Qué es Claude y cómo convierte tus palabras en trabajo

Introducción

Mucha gente usa Claude como un buscador con el que se conversa. Eso desperdicia casi todo su valor. Claude es un sistema de IA generativa que trabaja con lenguaje, razonamiento, archivos, imágenes y código, y que — cuando se le da contexto y estructura— puede planificar y ejecutar tareas, no solo responder preguntas.

Objetivos de aprendizaje

- Explicar qué es Claude y qué tipo de problema resuelve.
- Entender, en términos simples, cómo pasa de un prompt a una respuesta o a un plan.
- Reconocer sus límites para no usarlo mal.

Conceptos clave

- **Modelo de lenguaje grande (LLM):** sistema entrenado con enormes cantidades de texto que aprende a predecir y generar lenguaje coherente. No "consulta una base de datos de verdades": genera la respuesta más plausible dado tu contexto e instrucciones.
- **Contexto:** todo lo que Claude "tiene a la vista" en una conversación (tu mensaje, archivos adjuntos, instrucciones previas, resultados de herramientas). La calidad de la salida depende muchísimo de la calidad del contexto.
- **Generación, no recuperación:** Claude no "sabe la verdad"; produce respuestas a partir de patrones, contexto y capacidades disponibles. Por eso puede equivocarse con seguridad (alucinaciones).
- **De prompt a plan:** ante una tarea compleja, Claude puede descomponerla en pasos, usar herramientas (búsqueda, código, archivos) y encadenar acciones hasta entregar un resultado.

Desarrollo

Imagina que le pides a Claude: *"Organiza estas 40 facturas en una tabla y dime cuánto gasté por proveedor."* Internamente ocurre algo parecido a esto:

1. **Interpreta el objetivo:** entiende que quieres una tabla y un agregado por proveedor.
2. **Evalúa el contexto disponible:** ¿están las facturas adjuntas? ¿en qué formato?
3. **Decide un enfoque:** quizá leer los archivos, extraer campos, sumar por proveedor.
4. **Ejecuta** (en entornos con herramientas, como Cowork o Code, puede leer archivos y correr código; en Chat trabaja con lo que le des en contexto).
5. **Compone la respuesta** en el formato pedido.

La lección central: **Claude rinde en proporción a la claridad con la que defines qué quieres, con qué información, bajo qué límites y en qué formato.** Una pregunta vaga produce una respuesta vaga.

Lo que Claude **no** hace:

- No reemplaza tu juicio en decisiones críticas (legales, médicas, financieras, estratégicas).
- No "sabe" hechos posteriores a su fecha de entrenamiento sin buscar en la web.
- No garantiza exactitud en cálculos o citas: hay que verificar.

Ejemplo guiado

Prompt pobre:

Háblame de la inteligencia artificial.

Resultado esperado: un texto genérico, largo, que no sirve para nada concreto.

Prompt con criterio:

Explícame qué es la inteligencia artificial generativa a una persona adulta sin formación técnica que quiere entender de qué se habla en las noticias.

- Máximo 250 palabras.
- Usa 2 analogías cotidianas.
- Termina con 3 ideas falsas comunes y por qué son falsas.

Resultado esperado: una explicación a medida, útil y verificable.

Ejercicio

Toma tres preguntas vagas que harías normalmente (por ejemplo "ayúdame con mi CV", "resume esto", "dame ideas de marketing") y reescríbelas especificando: objetivo, audiencia, restricciones y formato de salida.

Desarrollo del ejercicio

Ejemplo con "ayúdame con mi CV":

Componente	Decisión
Objetivo	Mejorar el CV para una vacante concreta
Audiencia	Reclutador de una empresa de logística, lectura de 30 segundos
Restricciones	Una página, sin inventar logros, conservar mis fechas
Formato	CV reescrito + lista de 5 cambios y por qué

Prompt resultante:

Actúa como reclutador senior de logística. Reescribe mi CV (lo pego abajo) para la vacante de "Coordinador de operaciones" (descripción también abajo).
Restricciones: una página, no inventes logros ni cifras, mantén mis fechas reales.
Entrega: (1) el CV reescrito y (2) una lista de los 5 cambios más importantes con una frase explicando por qué mejoran mis chances.
[CV] ...
[Vacante] ...

La diferencia no es "escribir más", sino **decidir tú** qué resultado quieres antes de pedirlo.

Errores frecuentes

- Pedir "hazlo bien" sin definir qué significa "bien".
- Asumir que Claude conoce tu contexto (tu empresa, tu público, tus datos) sin dárselo.
- Aceptar la primera respuesta sin revisarla.

Para llevar

Claude no adivina lo que quieres: **construye** lo que le pides. Tu trabajo es definir objetivo, contexto, límites y formato. Esa es la habilidad que multiplica todo lo demás.

Clase 1.2 — El ecosistema Claude: Chat, Proyectos, Code, Cowork y API

Introducción

"Claude" no es una sola cosa. Es una familia de entornos pensados para distintos tipos de trabajo. Elegir el entorno equivocado es la causa silenciosa de mucha frustración: gente intentando programar en una ventana de chat, o investigar sin proyectos donde acumular contexto.

Objetivos de aprendizaje

- Diferenciar Claude Chat, Proyectos, Claude Code, Cowork y la API.
- Saber cuándo conviene cada uno.
- Tomar la decisión con una sola pregunta guía.

Conceptos clave

- **Claude Chat:** la conversación interactiva (web, escritorio, móvil). Para pensar, escribir, analizar, estudiar y resolver dudas del día a día.
- **Proyectos:** espacios persistentes dentro de Chat con instrucciones, archivos y una base de conocimiento que Claude recuerda entre conversaciones.
- **Claude Code:** entorno de línea de comandos orientado a desarrollo de software: leer repositorios, editar código, ejecutar pruebas, refactorizar. Funciona como agente.
- **Claude Cowork:** agente de escritorio (en la app de Claude para Mac y Windows) que planifica y ejecuta tareas de varios pasos sobre tus archivos y aplicaciones locales, sin abrir terminal. Usa la misma arquitectura agéntica de Claude Code.
- **API / Claude Platform:** acceso programático para desarrolladores que quieren integrar Claude en sus propios productos.

Desarrollo

La pregunta que resuelve casi todas las dudas es: "**¿Qué tipo de trabajo estoy intentando hacer?**"

- ¿Necesitas **conversar y pensar** sobre un texto, una idea o un análisis puntual? → **Chat**.
- ¿Vas a volver una y otra vez al **mismo tema** con los mismos archivos e instrucciones? → **Proyectos**.
- ¿Trabajas sobre **código y repositorios** y quieres un agente técnico en tu terminal? → **Claude Code**.
- ¿Quieres que un agente **ejecute tareas sobre tus archivos y apps** del computador (ordenar carpetas, generar entregables, automatizar tareas recurrentes)? → **Cowork**.
- ¿Estás **construyendo un producto** que necesita IA por dentro? → **API**.

Claude Code y Cowork comparten ADN: ambos son **agénticos** (planifican y ejecutan varios pasos). La diferencia principal es la audiencia y el medio: Code vive en la terminal y es ideal para perfiles técnicos; Cowork vive en la app de escritorio y está pensado para que cualquiera automatice trabajo de conocimiento sin escribir código.

Ejemplo guiado

Una analista de marketing tiene estas tareas en una semana:

- Redactar un correo difícil → **Chat**.
- Mantener el "manual de marca" y generar copys consistentes durante meses → **Proyecto** con la guía de marca como conocimiento.
- Ordenar la carpeta de informes del trimestre y generar un resumen ejecutivo en PowerPoint → **Cowork**.
- Arreglar un bug en el script de scraping del equipo → **Claude Code**.

Ejercicio

Haz una lista de 10 tareas reales de tu trabajo o estudio. Para cada una, decide el entorno ideal y justifica en una frase.

Desarrollo del ejercicio

Ejemplo de matriz resuelta (parcial):

Tarea	Entorno	Por qué
Resumir un paper de 30 páginas	Chat	Tarea puntual, basta con adjuntar el PDF
Generar reportes mensuales con el mismo formato	Proyecto	Reúso de instrucciones y plantillas a lo largo del tiempo
Renombrar y clasificar 200 archivos	Cowork	Necesita actuar sobre archivos locales
Migrar funciones de un módulo	Claude Code	Trabajo técnico sobre el repositorio
Chatbot de soporte en mi web	API	Integración dentro de un producto propio

El criterio se entrena: a las pocas semanas eliges entorno casi sin pensar.

Errores frecuentes

- Usar Chat para todo, incluso cuando el trabajo pedía continuidad (Proyectos) o ejecución (Cowork/Code).
- No aprovechar Proyectos y repetir el mismo contexto en cada conversación.
- Esperar que Chat actúe sobre tus archivos locales como lo hace Cowork.

Para llevar

Primero decide **el tipo de trabajo**, luego el entorno. Chat para pensar, Proyectos para continuidad, Code para programar, Cowork para ejecutar, API para construir.

Clase 1.3 — Elegir modelo: Haiku, Sonnet y Opus

Introducción

Claude ofrece varios modelos con distinto equilibrio entre **velocidad, costo y capacidad de razonamiento**. Elegir bien ahorra tiempo y dinero, y mejora resultados. La trampa común es pensar que "el más potente siempre es mejor": no lo es, igual que no usas un camión para ir a comprar el pan.

Objetivos de aprendizaje

- Conocer los tres niveles de modelo y para qué brilla cada uno.
- Elegir modelo según la naturaleza de la tarea.
- Combinar modelos en flujos más sofisticados.

Conceptos clave

- **Haiku** (p. ej. Haiku 4.5): el más rápido y económico. Excelente para tareas simples, repetitivas y de alto volumen: clasificar, extraer, formatear, enrutar, resúmenes breves.
- **Sonnet** (p. ej. Sonnet 4.6): el equilibrado. Razonamiento sólido para el trabajo cotidiano: programar, escribir, analizar, investigar. Es el **modelo por defecto** para la mayoría de los casos.
- **Opus** (p. ej. Opus 4.6): el especialista en razonamiento profundo. Para problemas novedosos, decisiones de arquitectura, planificación extensa y tareas que Sonnet resuelve mal.
- **Ventana de contexto**: cuánta información cabe "a la vista". Los modelos mayores admiten ventanas muy grandes (hasta ~1 millón de tokens en Opus/Sonnet, aprox. 1.500 páginas), útil para codebases enteras o documentación masiva.

Desarrollo

Una forma práctica de decidir es responder cuatro preguntas:

1. **¿La tarea necesita velocidad o profundidad?** Velocidad → Haiku. Profundidad → Opus. Punto medio → Sonnet.
2. **¿La salida la leerá un humano o alimentará a un sistema automatizado?** Si es parte de un pipeline de alto volumen, Haiku suele bastar.
3. **¿Importa mucho el costo?** A mayor volumen, más conviene el modelo suficiente, no el máximo.
4. **¿Requiere razonamiento largo o solo transformar texto?** Transformar (resumir, reformatear) → Haiku/Sonnet. Razonar entre muchas restricciones → Opus.

El criterio de oro: elige *el modelo suficiente para el problema*, no el más inteligente disponible.

Arquitectura combinada (avanzado): los sistemas más maduros usan los tres juntos. Haiku como "router" que clasifica y resuelve lo simple; Sonnet para el grueso del trabajo medio; Opus para el 10–15 % de casos que exigen razonamiento profundo.

Ejemplo guiado

Tarea: procesar 5.000 reseñas de clientes para (a) clasificarlas por sentimiento y (b) escribir un informe estratégico de mejoras.

- (a) Clasificación de 5.000 textos → **Haiku** (rápido y barato, tarea repetitiva).

- (b) Informe estratégico que sintetiza patrones y propone decisiones → **Opus** o **Sonnet** según profundidad requerida.

Resultado: pagas poco por el volumen y reservas la potencia para donde aporta.

Ejercicio

Clasifica estas 8 tareas en Haiku / Sonnet / Opus: (1) corregir ortografía de un correo, (2) decidir la arquitectura de un sistema nuevo, (3) extraer fechas de 1.000 contratos, (4) escribir un artículo de blog, (5) traducir un menú, (6) planificar un proyecto de 6 meses con dependencias, (7) resumir una reunión, (8) refactorizar un módulo crítico interdependiente.

Desarrollo del ejercicio

Tarea	Modelo	Razón
1. Corregir ortografía	Haiku	Transformación simple
2. Arquitectura de sistema nuevo	Opus	Razonamiento entre muchas restricciones
3. Extraer fechas de 1.000 contratos	Haiku	Extracción repetitiva de alto volumen
4. Artículo de blog	Sonnet	Escritura de calidad, trabajo cotidiano
5. Traducir un menú	Haiku	Transformación directa
6. Planificar proyecto de 6 meses	Opus	Planificación extensa con dependencias
7. Resumir una reunión	Haiku/Sonnet	Síntesis breve; Sonnet si requiere matiz
8. Refactor de módulo crítico	Opus	Cambios acoplados que exigen razonar el impacto

Errores frecuentes

- Usar siempre el modelo más potente "por si acaso" (lento y caro).
- Usar el más rápido para tareas que sí necesitan razonamiento (resultados pobres).
- No probar: a veces Sonnet resuelve perfecto lo que creías necesitaba Opus.

Para llevar

Haiku para volumen y velocidad, Sonnet por defecto, Opus para lo difícil. Empieza por el suficiente y sube solo si la calidad lo pide.

Clase 1.4 — Configuración, multiplataforma, hábitos y seguridad

Introducción

Empezar bien evita problemas después. Esta clase no trata de "dónde está el botón", sino de adquirir hábitos profesionales desde el día uno: cómo organizar el trabajo, qué no subir nunca, y cómo asegurar que las salidas de Claude se revisan antes de usarse en serio.

Objetivos de aprendizaje

- Configurar tu uso de Claude de forma ordenada y multiplataforma.
- Adoptar hábitos de privacidad y manejo de datos sensibles.
- Establecer la regla de revisión humana en tareas críticas.

Conceptos clave

- **Acceso multiplataforma:** Claude está disponible en web, escritorio (Mac/Windows) y móvil. Funciones como Cowork requieren la app de escritorio y un plan de pago.
- **Datos sensibles:** información personal, confidencial o regulada que no debería compartirse sin autorización.
- **Revisión humana (human-in-the-loop):** la práctica de que una persona valide la salida antes de su uso real, especialmente en lo crítico.
- **Higiene de espacios:** separar pruebas, material interno y entregables finales.

Desarrollo

Una configuración profesional mínima incluye:

1. **Cuenta y plan adecuados:** elige según necesidades (las funciones de agente y conectores avanzados suelen requerir plan de pago).
2. **Organización:** crea proyectos por dominio (un proyecto por área o cliente), nómbralos con claridad y no mezcles temas incompatibles.
3. **Política de datos:** define qué información puede y no puede entrar a la herramienta. Ante la duda, anonimiza.
4. **Revisión:** decide quién revisa qué. Nada legal, médico, financiero o estratégico se usa sin validación humana.
5. **Reutilización:** guarda tus mejores instrucciones y plantillas; no reinventes la rueda cada vez.

Ejemplo guiado

Política mínima de uso responsable (una página) para un equipo pequeño:

1. No subir datos personales de clientes sin anonimizar.
2. No pegar credenciales, contraseñas ni secretos.
3. Marcar todo entregable generado con IA como "borrador" hasta su revisión.
4. Un proyecto por cliente; archivos nombrados como AAAA-MM_tema.
5. Revisión obligatoria de cifras, citas y afirmaciones legales/financieras.
6. Registrar qué prompts y skills funcionan en un documento compartido.

Ejercicio

Redacta una política mínima de uso responsable de Claude (máximo una página) adaptada a tu contexto (estudio personal, equipo o empresa).

Desarrollo del ejercicio

Una buena política responde a cinco preguntas: **qué** datos están permitidos, **qué** está prohibido, **quién** revisa, **cómo** se organizan los espacios y **dónde** se guarda lo que funciona. Si tu contexto es personal, simplifícala:

por ejemplo, "no subo documentos de terceros sin permiso, verifico toda cifra antes de usarla, y guardo mis prompts buenos en un archivo". Lo importante es que exista y la sigas.

Errores frecuentes

- Subir información confidencial "porque era más rápido".
- Mezclar pruebas y entregables en el mismo espacio y perder el control.
- Confiar ciegamente en salidas sin revisar lo crítico.

Para llevar

La seguridad y el orden no son burocracia: son lo que te permite usar Claude en serio sin arrepentirte. Define qué entra, quién revisa y dónde guardas lo que funciona.

PILAR 2 — Claude Chat: conversar con criterio

El chat es la puerta de entrada y, bien usado, una de las herramientas más potentes. Este pilar te lleva de "hacer preguntas" a "diseñar interacciones": prompting profesional, control de la salida, iteración disciplinada y las capacidades de investigación (búsqueda web, pensamiento extendido y deep research).

Clase 2.1 — Anatomía de un prompt profesional

Introducción

Un prompt es una instrucción, pero un *buen* prompt es un encargo bien hecho. La diferencia entre resultados mediocres y excelentes casi nunca está en el modelo: está en cómo se formula la petición.

Objetivos de aprendizaje

- Identificar los componentes de un prompt efectivo.
- Aplicar una plantilla reutilizable.
- Reconocer por qué especificar el tema no basta.

Conceptos clave

Un prompt robusto suele incluir, de forma explícita o implícita, ocho componentes:

1. **Rol** — desde qué perspectiva debe responder ("actúa como editor de estilo").
2. **Objetivo** — el resultado concreto que buscas.
3. **Contexto** — información de fondo relevante.
4. **Entrada** — los datos o material a procesar.
5. **Restricciones** — límites (longitud, tono, qué no hacer).
6. **Criterios de calidad** — qué debe cumplir para ser buena.
7. **Formato de salida** — tabla, lista, JSON, informe, etc.
8. **Referencia o ejemplo** — un modelo de cómo se ve "bien".

Desarrollo

No todos los prompts necesitan los ocho componentes, pero los buenos rara vez omiten **objetivo, contexto, restricciones y formato**. El error más común es indicar el **tema** ("escribe sobre liderazgo") sin indicar el **resultado** ("escribe un correo de 150 palabras para anunciar a mi equipo un cambio de liderazgo, tono cercano y honesto, sin sonar corporativo").

Plantilla base reutilizable:

```
Actúa como [rol].
Tu objetivo es [resultado concreto].
Contexto: [información relevante].
Entrada: [datos / material].
Restricciones: [longitud, tono, qué evitar].
Criterios de calidad: [qué debe cumplir].
Formato de salida: [tabla / lista / informe / JSON...].
Antes de responder, dime si falta información crítica.
```

La última línea ("dime si falta información crítica") es un pequeño truco poderoso: convierte a Claude en un colaborador que pregunta en vez de adivinar.

Ejemplo guiado

Antes:

```
Resume este informe.
```

Después:

```
Actúa como analista. Resume el informe adjunto para un comité directivo que tiene 3 minutos. Entrega: (1) un titular de una línea, (2) 3 hallazgos clave en viñetas, (3) una recomendación. Máx. 180 palabras, sin jerga. Si el informe no respalda alguna afirmación, márcala como "a verificar".
```

Ejercicio

Rediseña cinco prompts que uses habitualmente aplicando la plantilla. Para cada uno, anota qué componente le faltaba antes.

Desarrollo del ejercicio

Toma por ejemplo "dame ideas para un post". El componente ausente suele ser el **formato** y los **criterios de calidad**. Versión mejorada:

Actúa como community manager. Objetivo: 5 ideas de post de LinkedIn para una consultora de datos que quiere atraer pymes. Criterios: cada idea debe partir de un dolor real del cliente y prometer un beneficio concreto. Formato: tabla con columnas Idea | Gancho | Llamado a la acción. Tono profesional pero humano.

Al hacer este ejercicio descubrirás que casi siempre falta lo mismo: formato y criterios de calidad. Corregir eso, por sí solo, eleva la calidad de tus salidas.

Errores frecuentes

- Especificar el tema pero no el resultado.
- Omitir el formato y luego pelearse con la estructura de la respuesta.
- Dar por hecho un contexto que Claude no tiene.

Para llevar

Un buen prompt es un encargo claro: rol, objetivo, contexto, restricciones, criterios y formato. Empieza por el resultado que quieres, no por el tema.

Clase 2.2 — Patrones avanzados de prompting

Introducción

Una vez dominas la anatomía básica, hay patrones repetibles que resuelven problemas concretos: obtener estructura antes que contenido, forzar autocrítica, adaptar a varias audiencias. Conocerlos es como tener atajos profesionales.

Objetivos de aprendizaje

- Aplicar patrones de prompting según la situación.
- Combinar patrones para tareas complejas.

Conceptos clave — catálogo de patrones

- **Estructura primero, detalle después.**
- **Salida con control de calidad** (sección de verificación).
- **Adaptación por audiencia.**
- **Prompt por etapas** (no resuelvas todo de una vez).
- **Revisión crítica** (pídele que critique su propia respuesta).
- **Comparación con criterios definidos.**
- **Few-shot** (dar ejemplos de entrada→salida).
- **Cadena de razonamiento** (pedir que piense paso a paso antes de concluir).

Desarrollo

Patrón 1 — Estructura primero:

No resuelvas todavía. Primero propón un esquema en 6 puntos.
Cuando te diga "ok", desarrolla cada punto con un ejemplo.

Útil para informes, cursos, propuestas: evita rehacer todo cuando la estructura no era la que querías.

Patrón 2 — Control de calidad incorporado:

Redacta la respuesta. Luego añade una sección "Verificación" con:

- supuestos que hiciste,
- posibles errores,
- qué dato adicional mejoraría la precisión.

Patrón 3 — Adaptación por audiencia:

Explica [concepto] para: (1) un niño de 12 años, (2) un gerente sin perfil técnico,
(3) un especialista. Usa una tabla comparativa de las tres versiones.

Patrón 4 — Few-shot: muéstrale 2-3 ejemplos del formato exacto que quieres y pídele que siga el patrón. Es la forma más fiable de obtener salidas consistentes.

Ejemplo guiado

Combinando patrones para una propuesta comercial:

Etapas 1: propón la estructura de la propuesta en 7 secciones (no la escribas aún).
Etapas 2 (cuando apruebe): desarrolla cada sección.
Etapas 3: añade una "Verificación" con riesgos del proyecto y supuestos.

Ejercicio

Toma un mismo contenido (por ejemplo, explicar qué es una hipoteca) y genera tres versiones para tres audiencias usando el patrón de adaptación. Luego aplica el patrón de control de calidad a una de ellas.

Desarrollo del ejercicio

La clave del patrón de audiencia es que **cambia el vocabulario y los ejemplos, no los hechos**. Para un niño: "es un préstamo grande para comprar una casa que pagas poquito a poquito durante muchos años". Para un gerente: foco en flujo de caja y tasa. Para un especialista: amortización, TAE, riesgo de tasa variable. Al pedir luego la "Verificación", Claude debería señalar supuestos como "asumo tasa fija" o "las cifras de ejemplo son ilustrativas, no asesoría financiera" — exactamente el tipo de transparencia que buscas.

Errores frecuentes

- Pedir todo de golpe cuando convenía ir por etapas.
- No usar few-shot cuando la consistencia de formato importa.
- Olvidar pedir autocritica en tareas donde el error es costoso.

Para llevar

Los patrones son herramientas: estructura primero para evitar retrabajo, verificación para reducir errores, audiencia para comunicar, few-shot para consistencia.

Clase 2.3 — Controlar forma, profundidad y tono

Introducción

A veces la respuesta es correcta pero inservible: demasiado larga, demasiado técnica, en el formato equivocado. Controlar la *forma* de la salida es una habilidad propia.

Objetivos de aprendizaje

- Dirigir formato, longitud, profundidad y tono.
- Pedir variantes y elementos de control (supuestos, contraejemplos).

Conceptos clave

- **Formato:** tabla para comparar, pasos numerados para procesos, JSON para sistemas.
- **Profundidad graduada:** versión corta / media / larga.
- **Tono:** formal, cercano, técnico, divulgativo.
- **Elementos de control:** supuestos explícitos, contraejemplos, ejemplos buenos y malos.

Desarrollo

Técnicas que conviene tener siempre a mano:

- Pide **tablas** cuando hay comparación entre opciones o dimensiones.
- Pide **pasos numerados** cuando describes un proceso.
- Pide **ejemplos buenos y malos** para entender un criterio.
- Pide **supuestos explícitos** para descubrir vacíos.
- Pide **contraejemplos** para estresar una afirmación.
- Pide **tres longitudes** (corta para un tuit, media para un correo, larga para un informe) y elige.

Ejemplo guiado

```
Analiza esta política pública (adjunta). Entrega:  
1. Resumen ejecutivo de 120 palabras.  
2. Tabla "ventajas | riesgos".  
3. Explicación para un estudiante de secundaria.  
4. 5 preguntas críticas para un debate.
```

Una sola instrucción, cuatro formatos de salida, listos para distintos usos.

Ejercicio

Diseña un prompt "multiformato" sobre un tema de tu interés que produzca, en una sola respuesta, material listo para presentar (resumen + tabla + versión simple + preguntas).

Desarrollo del ejercicio

El secreto de un prompt multiformato es **numerar las entregas** y dar a cada una su propio formato y restricción de longitud. Si no numeras, Claude tiende a fundir todo en un bloque. Un buen patrón:

```
Sobre [tema], entrega exactamente estas 4 piezas, separadas por encabezados:  
A) Resumen de 100 palabras.  
B) Tabla de 5 filas con [dimensión 1] | [dimensión 2].  
C) Explicación en lenguaje simple (máx. 80 palabras).  
D) 5 preguntas para discutir.
```

Errores frecuentes

- No especificar longitud y recibir un muro de texto.
- Pedir "una tabla" sin decir qué columnas.
- Mezclar tono técnico y divulgativo sin querer.

Para llevar

La forma es parte del encargo. Di exactamente qué formato, qué longitud y qué tono; pide variantes cuando dudes.

Clase 2.4 — Iteración, depuración y evaluación de prompts

Introducción

Un prompt no se juzga por intuición, sino por resultados. Tratar el prompting como un proceso iterativo — probar, medir, ajustar— es lo que separa a quien "usa IA" de quien la domina.

Objetivos de aprendizaje

- Aplicar un método de iteración y depuración.
- Evaluar salidas con criterios objetivos.
- Guardar y versionar lo que funciona.

Conceptos clave

- **Iteración:** mejorar el prompt en ciclos cortos.
- **Criterios de evaluación:** precisión, claridad, utilidad, formato, consistencia y tiempo de corrección.
- **Versión ganadora:** el prompt que guardas porque resolvió bien la tarea.

Desarrollo

Método en seis pasos:

1. **Define el objetivo** y cómo sabrás que está logrado.
2. **Prueba** el prompt.
3. **Detecta fallas** concretas (¿qué falló y por qué?).
4. **Ajusta** una variable a la vez (contexto, restricción o formato).
5. **Compara** salidas de las variantes.
6. **Guarda** la versión ganadora en tu biblioteca.

Cambia **una sola cosa por iteración**: si modificas tres a la vez, no sabrás cuál ayudó.

Ejemplo guiado

Tarea: generar asuntos de correo para una newsletter. Pruebas tres variantes:

- V1: "Dame asuntos para mi newsletter." → genéricos.
- V2: añades audiencia y objetivo de apertura. → mejor, pero largos.
- V3: añades restricción "máx. 6 palabras, sin clickbait" + 2 ejemplos buenos. → ganadora.

Guardas V3 con una nota: "funciona por la restricción de longitud y los ejemplos".

Ejercicio

Toma una tarea real y crea tres variantes de prompt. Evalúalas con una tabla de criterios y declara una ganadora con su justificación.

Desarrollo del ejercicio

Una rúbrica simple de 1 a 5:

Criterio	V1	V2	V3
Precisión	3	4	5
Claridad	3	4	5
Formato correcto	2	3	5
Tiempo de corrección	bajo	medio	mínimo

La ganadora no es necesariamente la más elaborada, sino la que da mejor resultado con menos corrección posterior. Documenta **por qué** ganó: esa explicación es lo que te hará mejor prompter.

Errores frecuentes

- Juzgar prompts "a ojo" sin comparar.
- Cambiar muchas variables a la vez.
- No guardar lo que funcionó y reinventarlo cada semana.

Para llevar

Itera con método: una variable por vez, criterios claros y biblioteca de ganadores. El prompting es una habilidad medible, no magia.

Clase 2.5 — Búsqueda web con criterio

Introducción

Claude tiene conocimiento hasta su fecha de entrenamiento. Para hechos actuales —precios, noticias, quién ocupa un cargo— necesita **buscar en la web**. Pero buscar no es lo mismo que investigar: la clave es exigir trazabilidad y separar hechos de interpretaciones.

Objetivos de aprendizaje

- Saber cuándo activar y necesitar búsqueda web.
- Formular preguntas de investigación, no solo de resumen.
- Auditar la calidad de lo que devuelve.

Conceptos clave

- **Conocimiento con fecha de corte:** lo que el modelo "sabe" sin buscar tiene un límite temporal.
- **Búsqueda web:** capacidad de consultar internet en tiempo real para hechos actuales.
- **Trazabilidad:** poder rastrear cada afirmación hasta una fuente.
- **Hecho vs. interpretación vs. opinión:** distinción esencial al investigar.

Desarrollo

Activa o exige búsqueda cuando la respuesta dependa de información que cambia (precios, leyes vigentes, eventos recientes, datos de mercado) o de hechos posteriores al entrenamiento. Y cuando investigues, no pidas solo "resume": pide fuentes, consensos, controversias y vacíos.

Plantilla de investigación:

```
Investiga [tema] usando búsqueda web. Necesito:  
- ideas principales,  
- fuentes relevantes (con enlaces),  
- puntos de consenso,  
- controversias o desacuerdos,  
- riesgos de interpretación,  
- síntesis final.  
No inventes datos; si algo no está claro, márcalo como vacío.
```

Ejemplo guiado

Mal: "¿Cuánto cuesta el plan Pro de [servicio]?" sin pedir verificación → riesgo de dato desactualizado. Bien: "Busca en la web el precio actual del plan Pro de [servicio], cita la página oficial y la fecha de la consulta."

Ejercicio

Investiga un tema técnico actual de tu interés exigiendo fuentes, y luego pide a Claude que **audite su propia respuesta**: ¿qué afirmaciones carecen de fuente?, ¿hay contradicciones?

Desarrollo del ejercicio

La auditoría es el paso que casi nadie hace. Tras la investigación, escribe:

Revisa tu respuesta anterior. Lista: (1) afirmaciones sin fuente citada, (2) posibles datos desactualizados, (3) contradicciones entre fuentes, (4) qué buscarías para cerrar esos vacíos.

El resultado suele revelar que parte de lo que parecía "hechos" eran inferencias. Esa transparencia es exactamente lo que necesitas para usar la información con confianza.

Errores frecuentes

- Asumir que toda respuesta está actualizada.
- Aceptar afirmaciones sin fuente.
- Confundir "muchas fuentes" con "buenas fuentes".

Para llevar

Para hechos actuales, exige búsqueda y fuentes. Investigar es separar hecho de interpretación y auditar lo que recibes.

Clase 2.6 — Pensamiento extendido y razonamiento guiado

Introducción

Para problemas difíciles, Claude puede "pensar más" antes de responder: razonar paso a paso, sopesar alternativas, planificar. Esto se llama pensamiento extendido. Pero más razonamiento no siempre es mejor: en tareas simples, lo vuelve lento y enrevesado.

Objetivos de aprendizaje

- Reconocer cuándo una tarea se beneficia de razonamiento profundo.
- Pedir ese razonamiento de forma útil.
- Evitar el sobreanálisis.

Conceptos clave

- **Pensamiento extendido:** modo en que el modelo dedica más esfuerzo de razonamiento antes de dar la respuesta final.
- **Trade-off:** ganas profundidad a cambio de tiempo; conviene solo cuando el problema lo amerita.
- **Razonamiento guiado:** estructurar tú cómo debe pensar (criterios, etapas).

Desarrollo

Conviene en: problemas ambiguos, análisis estratégicos, diseño de soluciones, decisiones con compromisos (trade-offs) y planificación. No conviene en: tareas simples de transformación, donde solo añade ruido.

Técnica útil — comparar alternativas con criterios:

Analiza tres opciones para [problema]. Evalúa cada una por costo, complejidad, mantenimiento, riesgo y tiempo de implementación. Luego recomienda una y justifica.
Piensa paso a paso antes de concluir.

Ejemplo guiado

Decisión: ¿migrar a un nuevo proveedor de correo, mejorar el actual o externalizar? Pides análisis con criterios definidos y una recomendación. El razonamiento extendido aquí aporta porque hay múltiples factores en tensión.

Ejercicio

Plantea un caso real con al menos tres estrategias posibles y pide a Claude una matriz de decisión razonada con recomendación final.

Desarrollo del ejercicio

Una matriz de decisión típica puntúa cada opción por criterio y pondera:

Criterio (peso)	Opción A	Opción B	Opción C
Costo (30%)	...	...	...
Riesgo (25%)	...	...	...
Tiempo (25%)	...	...	...
Mantenimiento (20%)	...	...	...

Lo valioso no es el número final, sino que el razonamiento **explícite los supuestos y los pesos**. Si no estás de acuerdo con un peso, lo ajustas y vuelves a pedir el cálculo: ahí está el verdadero uso del pensamiento extendido.

Errores frecuentes

- Activar razonamiento profundo para todo y volver lento lo simple.
- Pedir "piensa bien" sin dar criterios.
- Aceptar la recomendación sin revisar los supuestos.

Para llevar

Usa pensamiento extendido para problemas con muchos factores en tensión; guía el razonamiento con criterios; no lo malgastes en lo trivial.

Clase 2.7 — Deep research como flujo de trabajo

Introducción

Cuando una pregunta exige revisar muchas fuentes y producir un informe sólido, no basta una búsqueda rápida. Deep research es un modo en que Claude investiga de forma extensa y autónoma, consultando cientos de fuentes y sintetizándolas. La habilidad clave es convertir eso en un **proceso reproducible**, no en un "investiga todo".

Objetivos de aprendizaje

- Entender qué es deep research y cuándo usarlo.
- Aplicar un flujo de investigación reproducible.
- Delimitar el alcance para obtener resultados útiles.

Conceptos clave

- **Deep research:** investigación extensa y multi-fuente que produce un informe estructurado con citas. Puede revisar cientos de fuentes en una sola tarea.
- **Alcance (scope):** los límites de la pregunta. Sin alcance, la investigación se dispersa.
- **Subpreguntas:** descomposición de la pregunta principal en partes investigables.

Desarrollo

Flujo sugerido en ocho pasos:

1. **Define la pregunta** principal con precisión.
2. **Delimita el alcance** (qué incluir y qué excluir, marco temporal, geografía).
3. **Identifica subpreguntas.**
4. **Recolecta** información de múltiples fuentes.
5. **Contrasta** fuentes entre sí.
6. **Organiza** la evidencia.
7. **Redacta** la síntesis.
8. **Revisa vacíos** y limitaciones.

El error capital es pedir "investiga todo sobre X". Compáralo con: "Investiga el impacto de la jornada laboral de 4 días en productividad y rotación, en empresas de tecnología, 2019–2025, países OCDE; quiero hallazgos, contradicciones y limitaciones."

Ejemplo guiado

Pregunta amplia: "estado del mercado de vehículos eléctricos". Delimitada: "Evolución de precios y autonomía de autos eléctricos de gama media en Latinoamérica, 2022–2025, con fuentes y vacíos de datos". La segunda produce un informe accionable; la primera, un texto difuso.

Ejercicio

Aplica el flujo de ocho pasos a un tema relevante para ti. Entrega un informe corto con hipótesis, hallazgos y limitaciones.

Desarrollo del ejercicio

La parte más difícil —y más valiosa— es el paso 2. Antes de lanzar la investigación, escribe explícitamente: "Incluyo: ... / Excluyo: ... / Periodo: ... / Tipo de fuentes que confío: ...". Si entregas eso a Claude, el informe resultante será mucho más enfocado. Y termina siempre exigiendo una sección de **limitaciones**: qué no se pudo verificar, qué fuentes faltaron, qué conclusiones son tentativas. Un informe que reconoce sus límites es más confiable que uno que aparenta certeza total.

Errores frecuentes

- No delimitar el alcance.
- Confundir cantidad de fuentes con calidad.
- Omitir la sección de limitaciones.

Para llevar

Deep research brilla cuando lo conviertes en proceso: pregunta precisa, alcance delimitado, subpreguntas, contraste de fuentes y un informe que reconoce sus vacíos.

PILAR 3 — Trabajo con información: documentos, datos y entregables

Claude no solo conversa: lee PDFs e imágenes, interpreta hojas de cálculo, propone análisis y produce presentaciones y aplicaciones. Este pilar te enseña a convertir información en bruto en entregables útiles, sin perder de vista la verificación.

Clase 3.1 — Análisis de documentos e imágenes

Introducción

Una de las capacidades más prácticas de Claude es leer y razonar sobre material multimodal: contratos, informes, artículos, capturas de pantalla, gráficos, fotos de texto. Bien usada, ahorra horas de lectura y extracción manual.

Objetivos de aprendizaje

- Aprovechar Claude para resumir, comparar y transformar documentos.
- Analizar imágenes con y sin texto.
- Mantener la verificación cuando el contenido es crítico.

Conceptos clave

- **Multimodal**: capacidad de procesar texto e imágenes en conjunto.
- **Extracción estructurada**: convertir texto libre en tablas o campos.
- **Comparación documental**: detectar diferencias y similitudes entre documentos.

Desarrollo

Casos de uso frecuentes: resumir PDFs largos, comparar dos contratos cláusula por cláusula, extraer obligaciones y fechas, convertir un informe en tabla de acciones, describir un gráfico, explicar una captura de pantalla, revisar un documento de políticas.

Prompt base para analizar un documento:

```
Analiza el documento adjunto. Entrega:  
- objetivo del documento,  
- ideas principales,  
- términos técnicos (con definición breve),  
- riesgos o ambigüedades,  
- resumen ejecutivo de 150 palabras,  
- tabla con acciones recomendadas (acción | responsable sugerido | prioridad).  
Marca como "a verificar" cualquier afirmación que el documento no respalde.
```

Para imágenes: pídele que primero **describa** lo que ve y luego **interprete**. Esto reduce errores: si la descripción es incorrecta, lo detectas antes de confiar en la interpretación.

Ejemplo guiado

Comparar dos versiones de un contrato:

```
Compara estas dos versiones del contrato (A y B). Entrega una tabla:  
cláusula | versión A | versión B | tipo de cambio (añadido/eliminado/modificado)  
| impacto para nosotros. Destaca los cambios que aumentan nuestro riesgo.
```

Ejercicio

Analiza dos documentos de naturaleza distinta (por ejemplo, uno académico y uno corporativo) y produce una comparativa de objetivos, estructura y calidad de la evidencia.

Desarrollo del ejercicio

La trampa de comparar documentos distintos es que Claude tiende a tratarlos como equivalentes. Para evitarlo, define **dimensiones comunes** de comparación antes de pedir el análisis: propósito, audiencia, tipo de evidencia, claridad, y limitaciones. Una tabla con esas dimensiones en las filas y los dos documentos en las columnas obliga a una comparación pareja. Termina pidiendo: "¿Cuál es más confiable para tomar una decisión y por qué?" — esa pregunta fuerza un juicio sintético, no solo descriptivo.

Errores frecuentes

- Confiar en la extracción de cifras sin verificarlas en el original.
- Pedir interpretación de una imagen sin pedir primero la descripción.
- Resumir un contrato y actuar sin revisión legal.

Para llevar

Claude lee y compara documentos e imágenes muy bien, pero en material crítico tú verificas. Pide descripción antes que interpretación, y extracción estructurada para reusar la información.

Clase 3.2 — Datos, hojas de cálculo y análisis

Introducción

Claude puede ayudarte a entender datos: explicar columnas, detectar problemas de calidad, proponer análisis y hasta ejecutar técnicas como agrupamiento (clustering) con K-means cuando trabaja en un entorno que corre código. Pero hay una línea roja: no delegar cálculos críticos sin validación.

Objetivos de aprendizaje

- Usar Claude para interpretar y diseñar análisis de datos.
- Entender K-means a nivel conceptual y cómo pedirlo.
- Saber qué no automatizar sin revisión.

Conceptos clave

- **Calidad de datos:** valores faltantes, duplicados, outliers, formatos inconsistentes.
- **Análisis exploratorio:** primera mirada para entender variables y relaciones.
- **K-means:** algoritmo que agrupa observaciones en k grupos (clusters) según su similitud. Útil para segmentar clientes, productos o comportamientos.
- **Validación humana:** revisar que los cálculos y supuestos tengan sentido de negocio.

Desarrollo

Lo que Claude hace bien con datos: explicar qué significa cada columna, detectar problemas de calidad, proponer preguntas de negocio, sugerir visualizaciones, interpretar resultados y redactar hallazgos. En entornos con ejecución de código (Cowork, Code, o análisis de archivos), también puede **correr el análisis** y devolver resultados y gráficos.

Lo que **no** debes hacer sin validar: confiar ciegamente en cálculos críticos, asumir que entendió perfectamente el dataset, o automatizar decisiones sin revisión.

Sobre **K-means** en términos simples: imagina puntos dispersos en un mapa. K-means elige k "centros", asigna cada punto al centro más cercano, recalcula los centros y repite hasta estabilizarse. El resultado son k grupos de puntos parecidos entre sí. Decidir el k correcto (cuántos grupos) es parte arte, parte método (p. ej., el "método del codo").

Prompt para un análisis exploratorio:

```
Analiza esta tabla (adjunta). Identifica:  
- variables clave y su tipo,  
- valores faltantes y posibles outliers,  
- 3 preguntas de negocio que podríamos responder,  
- visualizaciones sugeridas,
```

- una primera lectura ejecutiva.
- No saques conclusiones que los datos no respalden.

Prompt para segmentación:

Con estos datos de clientes (gasto mensual y frecuencia de compra), propón una segmentación con K-means. Explica cómo elegiste el número de grupos, describe cada segmento en lenguaje de negocio y sugiere una acción de marketing por segmento.

Ejemplo guiado

Una tienda quiere segmentar clientes. Claude propone 4 grupos: "ocasionales de bajo gasto", "frecuentes de gasto medio", "VIP de alto gasto" y "dormidos" (compraron mucho pero hace tiempo). Para cada uno, una acción: reactivar a los dormidos con una oferta, premiar a los VIP, etc. El valor no es el algoritmo: es la **traducción a decisiones**.

Ejercicio

Toma un archivo tabular (real o de ejemplo) y pide a Claude un mini análisis de negocio: preguntas, hallazgos y siguientes pasos. Si tienes un entorno con ejecución de código, pídele también una segmentación.

Desarrollo del ejercicio

El reporte breve ideal tiene tres partes: **preguntas** (qué queríamos saber), **hallazgos** (qué muestran los datos, con la salvedad de su calidad) y **siguientes pasos** (qué decisión o análisis sigue). Un error típico es saltar directo a "hallazgos" sin cuestionar la calidad: si hay 15% de valores faltantes en la columna de ingresos, cualquier conclusión sobre ingresos es frágil, y el informe debe decirlo. Pide siempre que Claude **anteponga las limitaciones de los datos** a las conclusiones.

Errores frecuentes

- Tratar el resultado de un cálculo como verdad sin revisar.
- Pedir clusters sin pedir su interpretación de negocio.
- Ignorar la calidad de los datos.

Para llevar

Claude es un gran copiloto de análisis: interpreta, propone y ejecuta, pero tú validas los cálculos críticos y traduces los resultados a decisiones.

Clase 3.3 — Presentaciones, informes y entregables

Introducción

El análisis solo vale si se comunica. Claude puede transformar hallazgos en presentaciones, informes ejecutivos, materiales didácticos o FAQs. La diferencia entre un entregable mediocre y uno excelente está, otra vez, en cuánto contexto y estructura le des.

Objetivos de aprendizaje

- Convertir análisis en comunicación clara.
- Pedir presentaciones e informes con estructura, audiencia y mensaje.
- Generar entregables en formatos reales (p. ej., PowerPoint, Word).

Conceptos clave

- **Mensaje central:** la idea única que el público debe recordar.
- **Una idea por diapositiva:** principio básico de buenas presentaciones.
- **Nota del presentador:** guion de apoyo para cada lámina.
- **Entregables en formato real:** en entornos con herramientas, Claude puede generar archivos .pptx, .docx, .xlsx o .pdf, no solo texto.

Desarrollo

La regla de oro: **no pidas "hazme una presentación"**. Pide estructura, audiencia, mensaje central, tono y evidencia. Compara:

Vago:

Hazme una presentación sobre nuestras ventas.

Profesional:

Convierte este análisis de ventas en una presentación de 8 diapositivas para el comité directivo. Mensaje central: "crecimos pero la rentabilidad cae por descuentos". Por diapositiva: un título-afirmación, 3 viñetas máximo, y una nota del presentador. Cierra con una recomendación. Tono ejecutivo, sin jerga.

En Cowork o entornos con la capacidad de crear archivos, puedes pedir directamente el .pptx o .docx terminado y descargarlo.

Ejemplo guiado

De un informe técnico de 20 páginas a material ejecutivo: pides primero el **esqueleto** (8 títulos-afirmación), lo revisas, y solo entonces pides desarrollar cada lámina. Así evitas rehacer toda la presentación si la narrativa no era la correcta.

Ejercicio

Convierte un análisis técnico tuyo en un esqueleto de presentación ejecutiva de 6–8 láminas, con título-afirmación y nota del presentador por lámina.

Desarrollo del ejercicio

La técnica clave es el **título-afirmación**: en vez de "Resultados del trimestre" (un tema), escribe "El margen cayó 4 puntos pese al alza de ventas" (una afirmación). Si lees solo los títulos de todas las láminas en orden, deberían contar la historia completa. Pide a Claude: "Que los títulos, leídos en secuencia, formen el argumento central". Esa sola instrucción transforma la calidad narrativa de la presentación.

Errores frecuentes

- Pedir la presentación sin definir el mensaje central.
- Saturar láminas de texto en vez de una idea por lámina.
- No revisar cifras antes de presentarlas.

Para llevar

Comunicar es parte del trabajo. Define audiencia y mensaje central, usa títulos-afirmación, y en entornos con herramientas genera el archivo final listo para usar.

Clase 3.4 — Artefactos: prototipos, miniapps y entregables interactivos

Introducción

Los artefactos son una capacidad de Claude para generar contenido visible y ejecutable —páginas web, calculadoras, dashboards, visualizaciones, pequeñas apps— que ves y pruebas al instante. Es la forma más rápida de pasar de una idea a algo tangible.

Objetivos de aprendizaje

- Entender qué es un artefacto y para qué sirve.
- Especificar bien antes de pedir una app.
- Iterar sobre prototipos.

Conceptos clave

- **Artefacto**: resultado interactivo o visual (HTML/CSS/JS, SVG, React, Markdown) que se renderiza junto a la conversación.
- **Prototipo**: versión funcional mínima para validar una idea.
- **Especificación**: definir usuario, tarea, entrada, salida y criterio visual antes de generar.

Desarrollo

Antes de pedir una app, define cinco cosas: **usuario** (quién la usa), **tarea** (qué resuelve), **entrada** (qué datos recibe), **salida** (qué devuelve) y **criterio visual/técnico** (cómo debe verse, qué restricciones). Saltarse esto produce prototipos genéricos que no sirven.

Casos de uso típicos: calculadoras (préstamos, notas, presupuestos), dashboards simples, páginas informativas, visualizaciones de datos, demos internas, herramientas educativas interactivas.

Prompt base:

Crea una miniapp web para [usuario]. Objetivo: [tarea].
Entradas: [campos]. Salida: [resultado].
Requisitos: interfaz clara y moderna, validación de entradas, todo en un solo archivo HTML. Primero descríbeme la solución en 4 líneas; si la apruebo, génerala.

Ejemplo guiado

"Calculadora de promedio escolar": el usuario es un estudiante, la tarea es saber si aprueba, la entrada son notas y pesos, la salida es el promedio y un estado aprobado/reprobado con color. Con eso definido, el prototipo sale útil a la primera.

Ejercicio

Crea un prototipo útil para tu contexto (aula, trabajo o personal). Especifica los cinco elementos antes de pedirlo y luego itera una mejora visual.

Desarrollo del ejercicio

La iteración es donde los artefactos brillan. Tras la primera versión, no rehagas todo: pide cambios concretos. "Agrega un botón para limpiar", "muestra el resultado en una tarjeta destacada", "valida que las notas estén entre 1 y 7". Cada petición específica mejora el prototipo sin romper lo que ya funcionaba. Guarda la versión final: muchas miniapps útiles nacen así, de iteraciones pequeñas sobre una buena especificación inicial.

Errores frecuentes

- Pedir "una app" sin definir usuario, entrada y salida.
- Rehacer todo en cada cambio en vez de iterar puntualmente.
- Pedir funciones que requieren un servidor cuando solo necesitabas algo local y simple.

Para llevar

Los artefactos convierten ideas en prototipos al instante. Especifica usuario, tarea, entrada, salida y criterio visual; luego itera con peticiones concretas.

PILAR 4 — Espacios persistentes: Proyectos, Estilos y Memoria

Hasta aquí cada conversación empezaba de cero. Este pilar enseña la **continuidad**: cómo hacer que Claude recuerde tu contexto, mantenga tu estilo y acumule conocimiento entre sesiones. Es el salto de "usar un chat" a "tener un espacio de trabajo".

Clase 4.1 — Proyectos y bases de conocimiento

Introducción

Un proyecto es un espacio persistente donde agrupas un objetivo, sus archivos, sus instrucciones y su conocimiento. En vez de re-explicar tu contexto en cada conversación, lo defines una vez y Claude lo tiene siempre presente. Es la herramienta que automatiza tareas repetitivas con calidad consistente.

Objetivos de aprendizaje

- Entender qué es un proyecto y su base de conocimiento.
- Diseñar un proyecto bien estructurado.
- Decidir qué va en instrucciones y qué en conocimiento.

Conceptos clave

- **Proyecto:** espacio persistente con instrucciones, archivos y conversaciones agrupadas por tema.
- **Base de conocimiento:** los documentos que subes al proyecto y que Claude consulta como contexto permanente (guías de marca, plantillas, manuales, datos de referencia).
- **Instrucciones del proyecto:** indicaciones que aplican a todas las conversaciones dentro de él (rol, tono, reglas).

Desarrollo

La diferencia entre instrucciones y conocimiento:

- **Instrucciones** = cómo comportarse ("actúa como mi asistente editorial, tono formal, siempre en español neutro, cita fuentes").
- **Conocimiento** = qué saber ("este es nuestro manual de marca, estas son las plantillas, estos los datos del último trimestre").

Buenas prácticas:

- **Un proyecto por dominio** (un cliente, un curso, un área). No mezcles temas incompatibles.
- **Instrucciones cortas y claras;** las largas se diluyen.
- **Archivos bien nombrados** en la base de conocimiento.
- **Revisión periódica** del contexto persistente: elimina lo desactualizado.

Ejemplo guiado

Proyecto "Newsletter mensual":

- Instrucciones: "Eres mi editor. Tono cercano y experto. Cada edición: 1 tema central, 3 enlaces comentados, un consejo práctico. Nunca inventes estadísticas."
- Conocimiento: ediciones anteriores, guía de estilo, lista de temas pendientes.
- Resultado: cada mes pides "prepara la edición de junio sobre X" y obtienes algo consistente con todo lo anterior, sin re-explicar nada.

Ejercicio

Diseña el "blueprint" de un proyecto Claude para una tarea recurrente tuya: define su objetivo, sus instrucciones y qué documentos irían en la base de conocimiento.

Desarrollo del ejercicio

Un blueprint sólido responde: (1) **Objetivo** — ¿qué produce este proyecto una y otra vez? (2) **Instrucciones** — rol, tono, reglas y formato por defecto. (3) **Conocimiento** — la lista concreta de archivos de referencia. (4) **Flujo típico** — el prompt que usarás cada vez. Por ejemplo, para un proyecto de soporte: objetivo = redactar respuestas a tickets; instrucciones = tono empático, máximo 120 palabras, siempre ofrecer un siguiente paso; conocimiento = base de FAQ y política de devoluciones; flujo = "responde este ticket: [texto]". Con eso, cada respuesta sale alineada sin repetir el contexto.

Errores frecuentes

- Meter todo en un solo proyecto gigante.
- Confundir instrucciones (comportamiento) con conocimiento (información).
- No actualizar la base de conocimiento y arrastrar datos viejos.

Para llevar

Los proyectos dan continuidad: define una vez el objetivo, las instrucciones y el conocimiento, y reutilízalo. Un proyecto por dominio, instrucciones cortas, conocimiento bien curado.

Clase 4.2 — Estilos de escritura personalizados

Introducción

Si necesitas que Claude escriba siempre con una voz consistente —tu marca, tu tono docente, tu registro técnico— los estilos personalizados lo logran. En vez de repetir "escribe formal pero cercano" cada vez, defines el estilo una vez y lo aplicas.

Objetivos de aprendizaje

- Entender qué es un estilo y cómo se define.
- Crear estilos para distintos propósitos.
- Mantener consistencia editorial.

Conceptos clave

- **Estilo personalizado:** un perfil de escritura reutilizable (tono, longitud, estructura, vocabulario permitido y prohibido).
- **Consistencia editorial:** que todo lo producido "suene" igual, sin importar quién lo pida.
- **Vocabulario permitido/prohibido:** palabras o expresiones que sí o no usar.

Desarrollo

Un estilo bien definido especifica: **tono** (formal, cercano, técnico), **nivel técnico** (define términos o asume conocimiento), **longitud** típica, **estructura** preferida, **vocabulario a usar** y **vocabulario a evitar**, y **cómo presentar ejemplos**.

La mejor forma de crear un estilo es darle a Claude **muestras de texto** que representen la voz deseada y pedirle que extraiga el patrón, o describirlo explícitamente.

Ejemplo de estilo "docente claro":

Escribe como docente que explica con claridad.

- Tono: cercano y preciso.
- Define cada término técnico la primera vez que aparece.
- Incluye un ejemplo práctico por concepto.
- Frases cortas. Evita la voz pasiva y los tecnicismos innecesarios.
- Cierra cada sección con una pregunta de reflexión.

Ejemplo guiado

Una consultora crea tres estilos: **académico** (citas, precisión, voz neutral), **ejecutivo** (directo, orientado a decisión, sin jerga) y **técnico** (preciso, con detalle de implementación). Según el destinatario, aplica uno u otro y el mismo contenido cambia de registro sin reescribirse desde cero.

Ejercicio

Crea tres estilos para tres propósitos distintos (por ejemplo, redes sociales, informes formales y explicaciones simples). Define para cada uno tono, longitud y vocabulario a evitar.

Desarrollo del ejercicio

La parte que más diferencia un estilo es el **vocabulario a evitar**. Para "redes sociales" quizá evites tecnicismos; para "informe formal", evites coloquialismos y signos de exclamación; para "explicación simple", evites toda palabra que requiera otra explicación. Prueba cada estilo con el **mismo** párrafo de entrada y compara las tres salidas: verás que la consistencia no se logra solo diciendo "formal", sino especificando qué incluir y qué excluir. Guarda los tres como estilos reutilizables.

Errores frecuentes

- Definir el tono con una sola palabra ("profesional") y esperar consistencia.
- No dar muestras de la voz deseada.
- Mezclar dos estilos en un mismo documento sin querer.

Para llevar

Los estilos garantizan voz consistente. Defínelos con tono, longitud, estructura y —sobre todo— vocabulario a usar y evitar. Da muestras para afinar.

Clase 4.3 — Memoria de Claude: cómo funciona y cuándo activarla

Introducción

La memoria permite a Claude recordar información tuya entre conversaciones: preferencias, contexto recurrente, datos útiles. Bien usada, ahorra repetición. Mal usada, arrastra contexto irrelevante o desactualizado. La clave es saber qué conviene que recuerde y qué no.

Objetivos de aprendizaje

- Entender qué es la memoria y en qué se diferencia de proyectos.

- Decidir cuándo activarla y qué guardar.
- Gestionar y depurar la memoria.

Conceptos clave

- **Memoria:** capacidad de retener información a través de conversaciones, según tus preferencias y permisos.
- **Diferencia con proyectos:** los proyectos acotan el contexto a un espacio temático; la memoria puede ser transversal a tu uso general.
- **Higiene de memoria:** revisar y depurar lo que Claude recuerda.

Desarrollo

Conviene que Claude recuerde cosas estables y útiles: tu rol y sector, tus preferencias de formato ("siempre respóndeme en viñetas y en español"), restricciones constantes, o el contexto de un trabajo en curso. **No** conviene guardar datos sensibles, información que cambia rápido, o contexto de un proyecto puntual que ya terminó.

Buenas prácticas:

- Activa la memoria cuando repitas el mismo contexto a menudo.
- Revisa periódicamente qué recuerda y elimina lo obsoleto.
- No confíes información sensible a la memoria.
- Si una respuesta sale "rara", revisa si la memoria está arrastrando contexto viejo.

Ejemplo guiado

Un periodista activa memoria para que Claude recuerde: "escribo para un medio de economía, tono riguroso, siempre cito fuentes, público general". A partir de ahí, no necesita repetirlo. Pero cuando empieza a cubrir otro tema, depura esa memoria para que no contamine el nuevo trabajo.

Ejercicio

Lista cinco datos que te ahorraría tiempo que Claude recordara, y cinco que **no** deberías guardar nunca. Justifica la separación.

Desarrollo del ejercicio

Ejemplos de lo que **sí** conviene: tu profesión, tu idioma y formato preferidos, tu nivel técnico, restricciones constantes (p. ej., "nunca uses emojis"), el objetivo de un proyecto en curso. Ejemplos de lo que **no**: contraseñas o datos financieros, información personal de terceros, datos que caducan (precios, agendas), y opiniones tuyas sobre personas concretas. El criterio: guarda lo **estable y no sensible**; mantén fuera lo **volátil o confidencial**. Esta disciplina evita el problema más común de la memoria: respuestas contaminadas por contexto que ya no aplica.

Errores frecuentes

- Guardar datos sensibles en memoria.
- No depurarla y arrastrar contexto desactualizado.
- Confundir memoria (transversal) con proyectos (acotados).

Para llevar

La memoria ahorra repetición cuando guarda lo estable y no sensible. Actívala con criterio, revísala con regularidad y mantén fuera lo confidencial o volátil.

PILAR 5 — Integración y extensión: MCP y Skills

Aquí Claude deja de ser una isla. MCP lo conecta con tus herramientas y datos (Notion, Canva, Slack, tu correo, tu base de datos); las Skills empaquetan capacidades reutilizables. Juntos transforman a Claude de "asistente que conversa" en "plataforma que opera dentro de tu trabajo".

Clase 5.1 — MCP y conectores: integra Claude con tus aplicaciones

Introducción

Un modelo aislado solo puede trabajar con lo que le pegas en el chat. MCP (Model Context Protocol) rompe ese límite: permite a Claude conectarse a tus aplicaciones y fuentes de datos para **leer y actuar** sobre ellas. Es la diferencia entre "ayúdame a redactar esto que copio de Notion" y "lee mi página de Notion, actualízala y avísame en Slack".

Objetivos de aprendizaje

- Entender qué problema resuelve MCP y cómo funciona.
- Conocer el modelo cliente-servidor de MCP.
- Usar conectores con criterio de seguridad.

Conceptos clave

- **MCP (Model Context Protocol):** un estándar abierto para conectar modelos de IA con herramientas y datos externos. Se le suele llamar "el USB-C de la IA": un conector universal.
- **Cliente y servidor MCP:** Claude actúa como **cliente**; las aplicaciones (Notion, Canva, Slack...) exponen sus capacidades a través de un **servidor** MCP. El cliente se conecta al servidor para usar sus herramientas.
- **Conector:** una integración concreta lista para activar (p. ej., el conector de Notion).
- **Herramientas (tools):** las acciones que un servidor expone (crear una página, buscar archivos, enviar un mensaje).
- **Permisos:** MCP respeta los permisos de la aplicación conectada; no los evade.

Desarrollo

La idea central: en vez de que cada app necesite una integración a medida con cada IA, MCP define un **formato común**. Una app implementa un servidor MCP una vez, y cualquier cliente compatible (como Claude) puede usarlo. Esto hace que el ecosistema crezca rápido: a mediados de 2026 el directorio de conectores supera las decenas de integraciones curadas, y el ecosistema más amplio, cientos.

Qué permite en la práctica:

- **Leer** datos de tus herramientas (tu workspace de Notion, tus diseños de Canva, tus mensajes).
- **Actuar**: crear o editar documentos, generar diseños, enviar mensajes, actualizar registros.

Ejemplos:

- **Notion**: Claude lee y escribe en tus páginas según tus permisos. Puedes pedir "resume las notas de este proyecto y crea una página con el plan".
- **Canva**: Claude genera diseños directamente en el lienzo, busca activos, exporta.

Riesgos a discutir (y gobernar):

- **Permisos excesivos**: dar más acceso del necesario. Concede el mínimo imprescindible.
- **Datos sensibles**: al conectar sistemas, parte de tu información entra en juego.
- **Acciones no deseadas**: un agente con permisos de escritura puede modificar cosas; revisa qué puede hacer.
- **Trazabilidad**: saber qué hizo, cuándo y por qué.
- **Dependencia de herramientas mal definidas**: un servidor mal hecho introduce errores.

Diseñar una integración con criterio significa preguntarse: ¿qué necesita leer?, ¿necesita escribir o solo leer?, ¿quién aprueba las acciones?, ¿cómo se audita?

Ejemplo guiado

Flujo "usuario → Claude → herramienta → resultado" para RR. HH.:

Usuario: "Revisa los CV nuevos en la carpeta de Drive y crea en Notion una ficha por candidato con nombre, experiencia y match con la vacante."
 Claude (cliente MCP): lee Drive (servidor MCP) → analiza CVs → crea páginas en Notion (servidor MCP) → devuelve un resumen con enlaces.

Permisos mínimos: leer la carpeta concreta de Drive, escribir solo en la base de datos de candidatos de Notion. Revisión humana antes de contactar a nadie.

Ejercicio

Diseña una integración hipotética para un área que conozcas (soporte, ventas, análisis). Dibuja en texto el flujo "usuario / Claude / herramienta(s) / resultado" y especifica qué permisos mínimos requiere y qué se revisa manualmente.

Desarrollo del ejercicio

Un buen diseño separa **lectura** de **escritura** y define un punto de control humano antes de cualquier acción irreversible. Por ejemplo, para soporte: usuario pide "clasifica los tickets de hoy y borra los duplicados". Lectura: la cola de tickets. Escritura: marcar duplicados — pero **borrar** debería requerir confirmación humana. El entregable ideal es un diagrama de cuatro nodos más una tabla de permisos: qué fuente, lectura o escritura, y quién aprueba. Si todo el flujo puede ejecutarse sin ningún control humano sobre acciones destructivas, el diseño es peligroso.

Errores frecuentes

- Conceder permisos amplios "para que funcione".
- No distinguir conectores de solo lectura de los que pueden modificar.
- Olvidar la trazabilidad y la revisión de acciones sensibles.

Para llevar

MCP conecta a Claude con tus herramientas mediante un estándar común (cliente-servidor). Da el mínimo permiso necesario, separa lectura de escritura y pon revisión humana donde haya acciones irreversibles.

Clase 5.2 — Skills personalizadas: capacidades reutilizables

Introducción

Cuando una organización madura su uso de IA, deja de depender de prompts sueltos e improvisados y empieza a construir **capacidades repetibles**. Eso son las Skills (Agent Skills): paquetes de instrucciones —y opcionalmente scripts y recursos— que enseñan a Claude a hacer una tarea concreta de forma consistente, una y otra vez.

Objetivos de aprendizaje

- Entender qué es una skill y por qué importa.
- Conocer la estructura de un archivo SKILL.md.
- Diseñar una skill propia.

Conceptos clave

- **Skill (Agent Skill):** una carpeta con un archivo **SKILL.md** que define una capacidad: qué hace, cuándo usarla y cómo. Puede incluir scripts, plantillas y documentación de apoyo.
- **Frontmatter:** la cabecera YAML del **SKILL.md** con metadatos obligatorios: **name** y **description**.
- **Divulgación progresiva (progressive disclosure):** principio de diseño por el cual Claude carga solo la parte de la skill que necesita, como un manual con índice, capítulos y apéndices.
- **Recursos empaquetados:** **scripts/** (código ejecutable), **references/** (documentación) y **assets/** (plantillas, archivos).

Desarrollo

Una skill se puede pensar, en términos sencillos, como **una unidad de trabajo reutilizable** con: propósito definido, entradas esperadas, reglas, formato de salida y criterio de calidad. Cuando Claude detecta que una tarea coincide con la **description** de una skill, la activa y sigue sus instrucciones.

Estructura de carpeta típica:

```
mi-skill/
├── SKILL.md           # instrucciones principales (el "cerebro")
├── scripts/           # scripts ejecutables (Python/Bash) opcionales
├── references/        # documentación que se carga si hace falta
└── assets/            # plantillas, imágenes, archivos base
```

Estructura mínima de un **SKILL.md**:

```
---
name: resumen-ejecutivo
description: Genera resúmenes ejecutivos de informes para comités directivos.
  Úsala cuando el usuario suba un informe largo y pida un resumen para directivos.
---

# Resumen ejecutivo

## Cuándo usar esta skill
Cuando haya que condensar un documento largo en una pieza breve para decisión.

## Proceso
1. Identifica objetivo, hallazgos y recomendación del documento.
2. Redacta un titular de una línea.
3. Lista 3-5 hallazgos clave.
4. Cierra con una recomendación accionable.

## Formato de salida
- Titular (1 línea)
- Hallazgos (viñetas)
- Recomendación (1 párrafo)
- Máximo 200 palabras. Marca como "a verificar" lo no respaldado por el documento.

## Errores a evitar
- No inventar cifras. No exceder la longitud. No usar jerga.
```

Buenas prácticas de diseño:

- La **description** es crítica: determina cuándo Claude activa la skill. Sé específico sobre qué hace y cuándo usarla.
- Mantén el **SKILL.md** **conciso** (idealmente bajo 5.000 palabras); si crece, divide el contenido en archivos de **references/** y enlázalos.
- Separa rutas que rara vez se usan juntas para ahorrar contexto.
- Los scripts y plantillas evitan que el prompt principal se infle.

Ejemplo guiado

Una skill "análisis-de-CVs":

- **description**: "Evalúa CVs contra una descripción de cargo. Úsala cuando el usuario suba CVs y una vacante y pida un ranking de candidatos."
- Proceso: extraer datos del CV → comparar con requisitos de la vacante → puntuar match → generar ficha y ranking.
- Salida: tabla candidato | match (%) | fortalezas | brechas | recomendación.
- Recursos: una plantilla de ficha en **assets/** y la rúbrica de evaluación en **references/**.

Una vez creada, cualquiera del equipo obtiene evaluaciones consistentes sin saber el prompt exacto.

Ejercicio

Diseña la ficha completa de una skill para una tarea repetitiva tuya: nombre, descripción (cuándo activarla), entradas, proceso, restricciones, formato de salida, casos de uso y errores comunes. Si te animas, escribe el `SKILL.md` con su frontmatter.

Desarrollo del ejercicio

La parte que la gente subestima es la `description`. "Hace resúmenes" es mala porque Claude no sabrá cuándo activarla. "Genera resúmenes ejecutivos de informes financieros trimestrales para el comité; úsala cuando el usuario suba un informe financiero y pida una versión para directivos" es buena porque describe **qué** hace y **cuándo** usarla. Escribe primero el proceso como si se lo explicaras a un colega nuevo: pasos numerados, claros, sin ambigüedad. Luego define el formato de salida con precisión (longitud, secciones, qué nunca hacer). Una skill bien escrita es, en el fondo, un procedimiento operativo (SOP) que Claude ejecuta de forma fiable.

Errores frecuentes

- Una `description` vaga: la skill no se activa cuando debería.
- `SKILL.md` enorme que desperdicia contexto en vez de usar `references/`.
- No definir el formato de salida ni los errores a evitar.

Para llevar

Las skills convierten prompts improvisados en capacidades repetibles. Lo esencial: una `description` precisa (qué y cuándo), un proceso claro, un formato de salida definido, y recursos separados para no inflar el contexto.

PILAR 6 — Claude como agente: Code, Cowork y operación real

Este pilar cierra el círculo: Claude no solo responde, **ejecuta**. Claude Code para trabajo técnico sobre repositorios, Cowork para automatizar trabajo de conocimiento en tu escritorio, y un proyecto integral que une todo lo aprendido. Termina con la lección más importante del curso.

Clase 6.1 — Claude Code: el agente para desarrollo

Introducción

Claude Code es un agente que vive en la terminal y trabaja sobre tu código: explora repositorios, explica, edita, prueba, refactoriza y documenta. No es autocompletado: es un colaborador que planifica y ejecuta tareas técnicas de varios pasos. Aunque no programes, entender qué hace te ayuda a saber cuándo recomendarlo o usarlo.

Objetivos de aprendizaje

- Entender qué es Claude Code y para qué brilla.
- Conocer su forma de trabajar como agente.
- Aplicar criterios de revisión al código generado.

Conceptos clave

- **Agente:** sistema que planifica y ejecuta múltiples pasos hacia un objetivo, usando herramientas (leer/editar archivos, correr comandos), no solo respondiendo.
- **Repositorio (repo):** el conjunto de archivos de un proyecto de software, normalmente con control de versiones (Git).
- **Refactorizar:** reorganizar código sin cambiar su comportamiento, para mejorarlo.
- **Revisión de cambios (diff):** ver exactamente qué líneas cambió antes de aceptarlas.

Desarrollo

Claude Code es ideal para: explorar un repositorio que no conoces ("explícame cómo funciona este proyecto"), explicar código complejo, refactorizar, escribir y correr pruebas, proponer arquitectura, depurar errores y documentar.

Trabaja en ciclo: entiende la tarea → explora los archivos relevantes → propone o aplica cambios → ejecuta pruebas → reporta. Como agente, puede encadenar muchos pasos, pero **tú mantienes el control**: revisa los diffs, no aceptes cambios masivos a ciegas, y pide pruebas asociadas a cada cambio.

Regla de oro técnica: **acepta cambios entendiéndolos**. Un agente puede tocar muchos archivos; aceptar sin revisar el impacto es la principal fuente de problemas.

Ejemplo guiado

Tarea: "Hay un bug: el cálculo de impuestos da mal con descuentos." Flujo típico:

1. Claude localiza la función de cálculo.
2. Explica por qué falla (orden de operaciones).
3. Propone el arreglo y muestra el diff.
4. Escribe una prueba que reproduce el bug y verifica que ahora pasa.
5. Tú revisas el diff y la prueba antes de aceptar.

Ejercicio

(Conceptual, no requiere programar.) Describe una tarea técnica de varios pasos y redacta cómo se la pedirías a un agente, incluyendo qué le exigirías revisar antes de aceptar cambios.

Desarrollo del ejercicio

Un buen encargo a un agente de código separa **objetivo**, **alcance** y **controles**. Por ejemplo: "Objetivo: añadir validación de email al formulario de registro. Alcance: solo el módulo de registro, no toques la base de datos. Controles: muéstrame el diff antes de aplicar, añade pruebas para emails válidos e inválidos, y no cambies el estilo de código existente." La diferencia entre un agente útil y uno peligroso no está en el agente, sino en si definiste alcance y controles. Pedir "arregla el registro" sin límites invita a cambios que no querías.

Errores frecuentes

- Aceptar cambios masivos sin leer el diff.
- No pedir pruebas que respalden los cambios.
- No acotar el alcance y permitir modificaciones fuera de lugar.

Para llevar

Claude Code es un agente técnico potente: explora, edita, prueba y documenta. Tú defines objetivo, alcance y controles, y revisas cada cambio antes de aceptarlo.

Clase 6.2 — Claude Cowork: tu agente de IA personal

Introducción

Cowork lleva la potencia agéntica de Claude Code a cualquier persona, sin terminal. Es un agente de escritorio dentro de la app de Claude que planifica y ejecuta tareas de varios pasos sobre tus archivos y aplicaciones locales. Le das un objetivo y te devuelve un entregable terminado.

Objetivos de aprendizaje

- Entender qué es Cowork y en qué se diferencia de Chat.
- Conocer sus capacidades (archivos locales, multipaso, plugins, MCP, tareas programadas).
- Diseñar tareas para delegarle con seguridad.

Conceptos clave

- **Cowork:** agente de escritorio (Mac y Windows, plan de pago) que ejecuta tareas complejas sobre tu computador: archivos locales, apps, varios pasos.
- **Misma arquitectura que Claude Code:** la potencia agéntica, pero accesible sin programar.
- **Subagentes:** para tareas complejas, Cowork puede coordinar varios agentes trabajando en paralelo.
- **Tareas programadas (cadencia):** escribes el prompt una vez, eliges frecuencia (diaria, semanal, mensual) y Claude lo ejecuta solo.
- **Dispatch:** conectas tu móvil a tu sesión de Cowork del escritorio; envías una tarea desde el teléfono, se ejecuta en el computador y recibes el resultado.
- **Plugins y MCP:** Cowork admite conectores MCP y plugins (paquetes de skills y herramientas).

Desarrollo

Cowork brilla cuando necesitas **ejecución**, no solo conversación: ordenar y renombrar archivos, leer una carpeta de informes y producir un resumen en PowerPoint, extraer datos de varios PDFs a una hoja de cálculo, preparar borradores recurrentes, o automatizar un reporte semanal.

La diferencia con Chat: en Chat tú haces; en Cowork delegas y Claude hace —sobre tus archivos reales— y te entrega el resultado. Por eso importa la **seguridad**: define qué carpeta puede tocar, revisa antes de acciones destructivas (borrar, sobrescribir) y trata el primer resultado como borrador.

Las **tareas programadas** son especialmente útiles: "cada lunes a las 8, revisa la carpeta de ventas y mándame un resumen de la semana anterior". Lo defines una vez y se ejecuta solo.

Ejemplo guiado

Encargo a Cowork:

En la carpeta "Informes Q2", lee los 12 PDFs, extrae de cada uno el ingreso total y el gasto total, arma una tabla en Excel con una fila por informe, y genera un resumen ejecutivo de una página en Word con las 3 conclusiones principales. No borres ni muevas ningún archivo original.

Cowork lee los PDFs, construye el Excel, redacta el Word y te entrega ambos archivos, dejando intactos los originales (porque se lo pediste).

Ejercicio

Diseña una tarea para delegar a Cowork que combine al menos tres pasos sobre archivos. Especifica el objetivo, los pasos, qué archivos puede tocar y qué **no** debe hacer.

Desarrollo del ejercicio

La cláusula más importante de tu encargo es la **restricción negativa**: qué no debe hacer. "No borres originales", "no modifiques archivos fuera de esta carpeta", "no envíes nada sin mostrármelo primero". Un agente competente seguirá tus límites si los pones; el problema surge cuando no los pones y el agente toma decisiones razonables pero no deseadas. Estructura tu encargo así: objetivo claro → pasos esperados → carpeta/archivos permitidos → restricciones negativas → formato del entregable. Después, ofrece programarla si es algo recurrente: ahí está el verdadero ahorro de tiempo.

Errores frecuentes

- Delegar acciones destructivas sin restricciones ni revisión.
- No especificar sobre qué archivos puede actuar.
- Usar Cowork para lo que bastaba un chat (y viceversa).

Para llevar

Cowork ejecuta trabajo real sobre tu computador. Delega con objetivo, pasos, archivos permitidos y restricciones negativas; revisa lo destructivo; y programa lo recurrente para que trabaje por ti.

Clase 6.3 — Proyecto integral: flujo completo de reclutamiento (caso BuenFit)

Introducción

Esta clase une todo lo aprendido en un caso realista, inspirado en el proyecto "BuenFit": construir un flujo de reclutamiento de principio a fin. Verás cómo se combinan prompting, análisis de documentos, proyectos, skills, MCP y agentes en una sola solución.

Objetivos de aprendizaje

- Integrar múltiples capacidades en un flujo de trabajo único.

- Diseñar un proceso con entradas, salidas, controles y medición.
- Aplicar el criterio "definir bien el problema" antes de automatizar.

Conceptos clave

- **Flujo de trabajo (workflow):** secuencia de pasos que transforma una entrada en un entregable.
- **Descriptor de cargo:** documento que define una vacante (responsabilidades, requisitos).
- **Punto de control humano:** etapa donde una persona aprueba antes de continuar.

Desarrollo

Imagina que BuenFit, una empresa, necesita cubrir varias vacantes y quiere estandarizar el reclutamiento. El flujo completo podría ser:

1. **Generar el descriptor de cargo.** A partir de una necesidad ("necesitamos un coordinador de operaciones"), Claude redacta un descriptor con responsabilidades y requisitos. *(Prompting + estilo de la empresa, desde un Proyecto.)*
2. **Crear la skill de descriptors.** Como esto se repetirá, se empaqueta como **skill** "generar-descriptor" para consistencia. *(Skills.)*
3. **Recibir y analizar CVs.** Claude lee los CVs (documentos/imágenes), extrae datos y los compara con el descriptor. *(Análisis de documentos.)*
4. **Puntuar y rankear candidatos.** Una tabla con match, fortalezas y brechas por candidato. *(Análisis + formato.)*
5. **Conectar las herramientas.** Vía **MCP**, los CVs entran desde Drive y las fichas se crean en Notion. *(MCP/conectores.)*
6. **Ejecutar el flujo.** **Cowork** orquesta los pasos sobre los archivos reales y produce los entregables. *(Agente.)*
7. **Control humano.** Antes de contactar a nadie, una persona revisa el ranking. *(Gobernanza.)*
8. **Medir.** Tiempo ahorrado por vacante, consistencia de las evaluaciones, calidad percibida. *(Valor.)*

Este es el patrón de un proyecto serio: cada capacidad del curso encaja en un eslabón, y el conjunto resuelve un problema real con controles y medición.

Ejemplo guiado

Prompt para el primer eslabón (dentro de un Proyecto con el estilo de la empresa):

Genera el descriptor de cargo para "Coordinador de Operaciones". Basándote en el manual de cargos de la base de conocimiento, incluye: misión del puesto, 5-7 responsabilidades, requisitos imprescindibles y deseables, y competencias. Tono alineado a nuestra marca empleadora. No inventes requisitos no habituales para este rol; si falta información, pregúntame.

Ejercicio

Diseña el flujo completo de una tarea recurrente de tu área (no tiene que ser reclutamiento). Identifica, para cada eslabón, qué capacidad de Claude usarías (prompting, documentos, proyectos, skills, MCP, agente) y dónde pondrías el control humano y la medición.

Desarrollo del ejercicio

El error más común al diseñar flujos es empezar por la herramienta ("usemos Cowork") en vez de por el problema. El orden correcto es: (1) define el **problema** y el entregable, (2) descompón en **pasos**, (3) asigna a cada paso la **capacidad** adecuada, (4) marca los **controles humanos** (sobre todo antes de acciones externas o irreversibles), (5) define cómo **medirás** el valor. Por ejemplo, un flujo de respuesta a reseñas: leer reseñas (MCP) → clasificar por sentimiento (Haiku) → redactar respuestas con el estilo de marca (skill) → un humano aprueba → publicar (MCP) → medir tiempo de respuesta y satisfacción. Fíjate cómo cada pilar del curso aparece en su lugar.

Errores frecuentes

- Automatizar antes de definir bien el problema.
- Quitar el control humano de las acciones sensibles para "ir más rápido".
- No medir, y por tanto no saber si el flujo realmente aporta.

Para llevar

Un flujo real combina todas las capacidades del curso en eslabones, con controles humanos y medición. Empieza por el problema, no por la herramienta.

Clase 6.4 — La clave no es Claude, eres tú

Introducción

La última lección es también la más importante de este manual: la productividad no aparece por usar una herramienta avanzada, sino por la claridad de quien la usa. Claude amplifica criterio; no lo reemplaza.

Objetivos de aprendizaje

- Interiorizar el principio "definir bien inputs y outputs".
- Consolidar una mentalidad de trabajo con IA.
- Saber cómo seguir mejorando después del curso.

Conceptos clave

- **Input:** lo que aportas (contexto, datos, objetivo, restricciones).
- **Output:** lo que esperas (formato, criterios de calidad).
- **Criterio:** el juicio humano que decide qué es bueno, qué se usa y qué se descarta.

Desarrollo

Todo el curso se puede comprimir en una idea: **los resultados mejoran cuando defines bien la entrada y la salida**. Quien sabe qué quiere, con qué información, bajo qué límites y en qué formato, obtiene de Claude un trabajo excelente. Quien improvisa, obtiene respuestas improvisadas.

La productividad real aparece cuando existe:

- **criterio** (saber qué es bueno),
- **proceso** (pasos claros y repetibles),

- **revisión** (validar antes de usar),
- **documentación** (guardar lo que funciona),
- y **buena definición del problema** (empezar por el resultado, no por la herramienta).

Claude no reemplaza la claridad humana: la amplifica. Una persona con criterio y una buena herramienta supera ampliamente a la misma herramienta usada sin método.

Ejemplo guiado

Dos personas piden lo mismo: "ayúdame con la estrategia de contenidos". La primera escribe esa frase y recibe generalidades. La segunda define audiencia, objetivos, recursos, restricciones de tiempo y formato esperado, y recibe un plan accionable. Misma herramienta, resultados opuestos. La diferencia eres tú.

Ejercicio

Escribe tu propio "manifiesto de trabajo con IA": cinco principios personales que seguirás al usar Claude, basados en lo aprendido en el curso.

Desarrollo del ejercicio

No hay una respuesta única, pero un buen manifiesto suele incluir variantes de estos cinco principios: (1) **Empiezo por el resultado**, no por el tema. (2) **Doy contexto suficiente**: la herramienta no adivina. (3) **Reviso lo crítico**: cifras, citas, lo legal y financiero. (4) **Guardo lo que funciona**: prompts, estilos y skills se vuelven biblioteca. (5) **Mido si vale la pena**: si no ahorra tiempo o mejora calidad, lo reconsidero. Hazlo tuyo, pégalo donde lo veas y revísalo en un mes: verás cuánto has crecido.

Errores frecuentes

- Creer que la herramienta hará el pensamiento por ti.
- Buscar el "prompt mágico" en vez de desarrollar criterio.
- Abandonar el método cuando hay prisa (justo cuando más importa).

Para llevar

La clave no es Claude, eres tú. Define bien entradas y salidas, mantén criterio, proceso y revisión, y la herramienta hará el resto. Esa es la habilidad que perdura aunque cambien las versiones.

ANEXOS DE REFERENCIA

Anexo A — Tablas comparativas rápidas

A.1 Entornos: cuándo usar cada uno

Entorno	Mejor para	Riesgo si se usa mal	Ejemplo correcto
Claude Chat	Pensar, escribir, analizar, estudiar, dudas puntuales	Quedarse en prompts vagos y respuestas superficiales	Síntesis comparativa con instrucciones claras

Entorno	Mejor para	Riesgo si se usa mal	Ejemplo correcto
Proyectos	Tareas recurrentes con contexto persistente	Mezclar temas incompatibles en un mismo espacio	Newsletter mensual con guía de estilo en conocimiento
Claude Code	Desarrollo de software sobre repositorios	Aceptar cambios sin revisar el diff	Refactor de un módulo con pruebas asociadas
Cowork	Ejecutar tareas multipaso sobre archivos/apps	Delegar acciones destructivas sin revisión	Extraer datos de PDFs a Excel sin tocar originales
API	Construir productos con IA integrada	Subestimar costos y verificación	Chatbot de soporte con modelo adecuado por tarea

A.2 Modelos: cuándo usar cada uno

Modelo	Fortaleza	Ideal para	Evitar para
Haiku	Velocidad y bajo costo	Clasificar, extraer, formatear, resúmenes breves, alto volumen	Razonamiento profundo y multifactor
Sonnet	Equilibrio (por defecto)	Programar, escribir, analizar, investigar	Casos que claramente exceden su razonamiento
Opus	Razonamiento profundo	Arquitectura, problemas novedosos, planificación extensa	Tareas simples (lento y caro)

A.3 MCP vs. Skills

	MCP / Conectores	Skills
Qué hace	Conecta Claude a herramientas y datos externos	Empaqueta una capacidad/proceso reutilizable
Analogía	El "USB-C" que enchufa apps	Un manual de procedimiento que Claude ejecuta
Da acceso a	Notion, Canva, Slack, Drive, etc.	Instrucciones, scripts y plantillas propias
Riesgo clave	Permisos excesivos, acciones no deseadas	Descripción vaga (no se activa), contexto inflado

Anexo B — Buenas prácticas transversales

1. **Define el resultado antes del contenido.** No empieces por el tema; empieza por el entregable.
2. **Da contexto suficiente.** Claude mejora mucho cuando entiende problema, audiencia y restricciones.
3. **Pide formato de salida.** Tabla, pasos, checklist, informe, JSON o resumen ejecutivo.
4. **Obliga a explicitar supuestos.** Evita respuestas que suenan seguras pero ocultan vacíos.
5. **Itera con método.** Una variable por vez; la primera respuesta rara vez es la mejor.
6. **Revisa lo sensible.** Nunca delegues a ciegas temas legales, financieros, médicos o estratégicos.
7. **Da el mínimo permiso.** Al conectar herramientas, concede solo lo necesario y revisa lo irreversible.

8. **Guarda lo que funciona.** Los mejores prompts se vuelven plantilla, estilo o skill.
 9. **Mide el valor.** Si no ahorra tiempo ni mejora calidad, reconsidéralo.
-

Anexo C — Plantillas listas para usar

C.1 Encargo general

Actúa como [rol]. Tu objetivo es [resultado].
Contexto: [información]. Entrada: [datos]. Restricciones: [límites].
Criterios de calidad: [qué debe cumplir]. Formato: [tabla/lista/informe/JSON].
Antes de responder, dime si falta información crítica.

C.2 Resumen ejecutivo

Analiza este material y produce un resumen ejecutivo. Entrega: contexto, hallazgos principales, riesgos, oportunidades, recomendaciones accionables y una conclusión de 5 líneas. No rellenes vacíos con supuestos no indicados.

C.3 Comparación de opciones

Compara [A], [B] y [C] por costo, complejidad, tiempo, riesgo, escalabilidad y mantenimiento. Entrega una tabla y luego una recomendación razonada con supuestos.

C.4 Revisión crítica de una respuesta

Evalúa la respuesta anterior. Detecta ambigüedades, afirmaciones débiles, partes incompletas y errores posibles. Luego reescribe una versión corregida.

C.5 Investigación con fuentes

Investiga [tema] con búsqueda web. Entrega ideas principales, fuentes con enlaces, consensos, controversias, riesgos de interpretación y síntesis. Marca los vacíos; no inventes datos.

C.6 Diseño de miniapp (artefacto)

Crea una miniapp web para [usuario]. Objetivo: [tarea]. Entradas: [campos]. Salida: [resultado]. Requisitos: interfaz clara, validación, un solo archivo HTML. Primero descríbeme la solución; si la apruebo, génerala.

C.7 Encargo a un agente (Cowork/Code)

Objetivo: [qué lograr]. Pasos esperados: [1, 2, 3].
Puede actuar sobre: [carpeta/archivos]. NO debe: [restricciones negativas].
Entregable: [formato]. Muéstrame el resultado antes de cualquier acción irreversible.

C.8 Esqueleto de SKILL.md

```
---  
name: nombre-de-la-skill  
description: Qué hace y CUÁNDO usarla (sé específico; esto decide su activación).  
---  
  
# Nombre de la skill  
## Cuándo usar  
## Proceso (pasos numerados)  
## Formato de salida  
## Errores a evitar
```

Anexo D — Glosario

- **Agente:** sistema que planifica y ejecuta varios pasos usando herramientas.
- **Artefacto:** salida visual/interactiva que se renderiza junto a la conversación.
- **Base de conocimiento:** documentos de referencia de un proyecto.
- **Contexto:** todo lo que Claude tiene a la vista al responder.
- **Conector:** integración lista de Claude con una app, vía MCP.
- **Deep research:** investigación extensa multi-fuente con informe citado.
- **Few-shot:** dar ejemplos de entrada→salida para guiar el formato.
- **Frontmatter:** cabecera YAML de un SKILL.md con **name** y **description**.
- **Ventana de contexto:** cantidad máxima de información (tokens) que el modelo maneja a la vez.
- **K-means:** algoritmo de agrupamiento en *k* grupos por similitud.
- **MCP (Model Context Protocol):** estándar abierto para conectar IA con herramientas y datos.
- **Pensamiento extendido:** modo en que el modelo razona más antes de responder.
- **Prompt:** la instrucción que das a Claude.
- **Skill (Agent Skill):** capacidad reutilizable definida en un SKILL.md.
- **Token:** unidad mínima de texto que procesa el modelo (≈ fragmento de palabra).
- **Trazabilidad:** poder rastrear cada afirmación o acción hasta su origen.

Anexo E — Proyecto final del curso

Elige una de las dos opciones y entrégala como un documento con su flujo, prompts, controles y medición.

Opción A — Asistente personal/estudio. Diseña un proyecto Claude para una asignatura o área de conocimiento: define instrucciones, base de conocimiento, al menos un estilo, una skill reutilizable, y un flujo

que use análisis de documentos. Incluye cómo verificarás la calidad.

Opción B — Flujo de trabajo profesional. Diseña un flujo asistido por Claude para un proceso real de tu trabajo (como el caso BuenFit): problema definido, entradas, pasos con la capacidad asignada (prompting, documentos, proyectos, skills, MCP, agente), puntos de control humano, restricciones de seguridad y métricas de valor.

Criterios de evaluación sugeridos: comprensión conceptual, calidad de los prompts, capacidad de iteración, calidad del análisis, uso correcto del formato de salida, pensamiento crítico, aplicación a un caso real, y documentación del proceso. Instrumentos: una demo o defensa breve explicando decisiones y resultados.

Anexo F — Cómo seguir aprendiendo

- Practica el ciclo iterar–evaluar–guardar hasta que sea automático.
 - Construye tu biblioteca personal de prompts, estilos y skills.
 - Mantente al día: los modelos y funciones cambian; revisa la documentación oficial periódicamente.
 - Enseña lo que aprendes: explicar un concepto es la mejor forma de dominarlo.
-

Fuentes y referencias

Datos de productos y capacidades verificados a mediados de 2026 con documentación y materiales oficiales de Claude/Anthropic y fuentes especializadas:

- [Choosing the right Claude model — Claude](#)
- [Models overview — Claude API Docs](#)
- [Equipping agents for the real world with Agent Skills — Anthropic](#)
- [Agent Skills — Claude Docs](#)
- [What is Model Context Protocol — Claude](#)
- [Get started with Claude Cowork — Claude Help Center](#)
- [Claude Cowork — Anthropic](#)
- [Connect Claude Code to tools via MCP — Claude Code Docs](#)

Verifica precios, disponibilidad y nombres de versión en la documentación oficial, ya que cambian con frecuencia. Los principios de método enseñados en este curso se mantienen aunque evolucionen las versiones.

Fin del curso. La clave no es Claude, eres tú.